

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue Sep 23 08:08:12 2025 from 10.51.10.70

```
(base) kolin@mosaic:~$ cd col7001/
(base) kolin@mosaic:~/col7001$ cd concurrency/
(base) kolin@mosaic:~/col7001/concurrency$ less sync
syncB.c sync.c syncL.c syncM.c
(base) kolin@mosaic:~/col7001/concurrency$ less syncL.c
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
```

Markdown pandoc 3.7... New folder
Monster

Notepad++ qccc-v3 ANUBHAV_

Hot days ahead
29°C

Q Search

Windows taskbar icons: File Explorer, Microsoft Store, Task View, Settings, Edge, Chrome, and system tray icons including network, volume, and date/time (09:04 23-09-2025).

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue Sep 23 08:08:12 2025 from 10.51.10.70

(base) kolin@mosaic:~\$ cd col7001/

(base) kolin@mosaic:~/col7001\$ cd concurrency/

(base) kolin@mosaic:~/col7001/concurrency\$ less sync

syncB.c sync.c syncL.c syncM.c

(base) kolin@mosaic:~/col7001/concurrency\$ less syncL.c

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$ less s

Markdown pandoc-3.7... New folder
Monitor



Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue Sep 23 08:08:12 2025 from 10.51.10.70

(base) kolin@mosaic:~\$ cd col7001/

(base) kolin@mosaic:~/col7001\$ cd concurrency/

(base) kolin@mosaic:~/col7001/concurrency\$ less sync

syncB.c sync.c syncL.c syncM.c

(base) kolin@mosaic:~/col7001/concurrency\$ less syncL.c

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$ less syncL

Markdown pandoc 3.7.1 New folder
Moniter



Notepad++



qecc-43



ANUSHAN

```
int counter = 0; // shared variable
```

```
void* increment(void* arg) {  
    for (int i = 0; i < 1000000; i++) {  
        counter++; // data race: multiple threads update at the same time  
    }  
    return NULL;  
}
```

```
int main() {  
    pthread_t t1, t2;  
    pthread_create(&t1, NULL, increment, NULL);  
    pthread_create(&t2, NULL, increment, NULL);  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
  
    printf("Final counter = %d \n", counter);  
    return 0;  
}
```

sync.c

MarkDown pandoc 3.7.1 New File
Monitor

Notepad++
gcc-e3
ANUBHAW


```
int counter = 0; // shared variable
```

```
void* increment(void* arg) {  
    for (int i = 0; i < 1000000; i++) {  
        counter++; // data race: multiple threads update at the same time  
    }  
    return NULL;  
}
```

```
int main() {  
    pthread_t t1, t2;  
    pthread_create(&t1, NULL, increment, NULL);  
    pthread_create(&t2, NULL, increment, NULL);  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
  
    printf("Final counter = %d \n", counter);  
    return 0;  
}
```

sync.c

Markdown pandoc 3.7... New folder
Monitor

Notepad++ qccc-43 ANUBHAV

Air Moderate
Now

Search

ENG
IN

09:04
23-09-2025

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue Sep 23 08:08:12 2025 from 10.51.10.70

(base) kolin@mosaic:~\$ cd col7001/

(base) kolin@mosaic:~/col7001\$ cd concurrency/

(base) kolin@mosaic:~/col7001/concurrency\$ less sync

syncB.c sync.c syncL.c syncM.c

(base) kolin@mosaic:~/col7001/concurrency\$ less syncL.c

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$

(base) kolin@mosaic:~/col7001/concurrency\$ less sync.c

(base) kolin@mosaic:~/col7001/concurrency\$ gcc sync.c

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

```
Last login: Tue Sep 23 08:08:12 2025 from 10.51.10.70
(base) kolin@mosaic:~$ cd col7001/
(base) kolin@mosaic:~/col7001$ cd concurrency/
(base) kolin@mosaic:~/col7001/concurrency$ less sync
syncB.c  sync.c  syncL.c  syncM.c
(base) kolin@mosaic:~/col7001/concurrency$ less syncL.c
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$ less sync.c
(base) kolin@mosaic:~/col7001/concurrency$ gcc sync.c
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Final counter = 1023471
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Final counter = 1018656
(base) kolin@mosaic:~/col7001/concurrency$ |
```

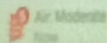
```
int counter = 0; // shared variable
pthread_mutex_t lock;

void* increment(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&lock);
        counter++; // data race: multiple threads update at the same time
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter = %d \n", counter);
    return 0;
}
```

syncL.c (END)



Q Search



09:06

23/09/2025

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

```
Last login: Tue Sep 23 08:08:12 2025 from 10.51.10.70
(base) kolin@mosaic:~$ cd col7001/
(base) kolin@mosaic:~/col7001$ cd concurrency/
(base) kolin@mosaic:~/col7001/concurrency$ less sync
syncB.c sync.c syncL.c syncM.c
(base) kolin@mosaic:~/col7001/concurrency$ less syncL.c
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$
(base) kolin@mosaic:~/col7001/concurrency$ less sync.c
(base) kolin@mosaic:~/col7001/concurrency$ gcc sync.c
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Final counter = 1023471
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Final counter = 1018656
(base) kolin@mosaic:~/col7001/concurrency$ less syncL.c
(base) kolin@mosaic:~/col7001/concurrency$ gcc syncL.c
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Final counter = 2000000
(base) kolin@mosaic:~/col7001/concurrency$ |
```



Q Search



Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <<http://gnu.org/licenses/gpl.html>>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<<https://www.gnu.org/software/gdb/bugs/>>.
Find the GDB manual and other documentation resources online at:
<<http://www.gnu.org/software/gdb/documentation/>>.

For help, type "help".

Type "apropos word" to search for commands related to "word"...

Reading symbols from a.out...

(gdb) list

```
1      #include <stdio.h>
2      #include <pthread.h>
3
4      int counter = 0; // shared variable
5
6      void* increment(void* arg) {
7          for (int i = 0; i < 1000000; i++) {
8              counter++; // data race: multiple threads update at the same time
9          }
10         return NULL;

```

(gdb)

```

5 void* increment(void* arg) {
6     for (int i = 0; i < 1000000; i++) {
7         counter++; // data race: multiple threads update at the same time
8     }
9     return NULL;
10

```

(gdb) p &counter

\$1 = (int *) 0x4014 <counter>

(gdb) disassemble increment

Dump of assembler code for function increment:

```

0x00000000000011a9 <+0>:      endbr64
0x00000000000011ad <+4>:      push    %rbp
0x00000000000011ae <+5>:      mov     %rsp, %rbp
0x00000000000011b1 <+8>:      mov     %rdi, -0x18(%rbp)
0x00000000000011b5 <+12>:     movl    $0x0, -0x4(%rbp)
0x00000000000011bc <+19>:     jmp     0x11d1 <increment+40>
0x00000000000011be <+21>:     mov     0x2e50(%rip), %eax      # 0x4014 <counter>
0x00000000000011c4 <+27>:     add     $0x1, %eax
0x00000000000011c7 <+30>:     mov     %eax, 0x2e47(%rip)      # 0x4014 <counter>
0x00000000000011cd <+36>:     addl    $0x1, -0x4(%rbp)
0x00000000000011d1 <+40>:     cmpl    $0xf423f, -0x4(%rbp)
0x00000000000011d8 <+47>:     jle     0x11be <increment+21>
0x00000000000011da <+49>:     mov     $0x0, %eax
0x00000000000011df <+54>:     pop     %rbp
0x00000000000011e0 <+55>:     ret

```

End of assembler dump.

(gdb) |

```
(gdb) p &counter  
$1 = (int *) 0x4014 <counter>  
(gdb) disassemble increment  
Dump of assembler code for function increment:
```

```
0x00000000000011a9 <+0>:    endbr64  
0x00000000000011ad <+4>:    push    %rbp  
0x00000000000011ae <+5>:    mov     %rsp, %rbp  
0x00000000000011b1 <+8>:    mov     %rdi, -0x18(%rbp)  
0x00000000000011b5 <+12>:   movl    $0x0, -0x4(%rbp)  
0x00000000000011bc <+19>:   jmp     0x11d1 <increment+40>    # 0x4014 <counter>  
0x00000000000011be <+21>:   mov     0x2e50(%rip), %eax      # 0x4014 <counter>  
0x00000000000011c4 <+27>:   add     $0x1, %eax  
0x00000000000011c7 <+30>:   mov     %eax, 0x2e47(%rip)  
0x00000000000011cd <+36>:   addl    $0x1, -0x4(%rbp)  
0x00000000000011d1 <+40>:   cmpl    $0xf423f, -0x4(%rbp)  
0x00000000000011d8 <+47>:   jle     0x11be <increment+21>  
0x00000000000011da <+49>:   mov     $0x0, %eax  
0x00000000000011df <+54>:   pop     %rbp  
0x00000000000011e0 <+55>:   ret
```

End of assembler dump.

```
(gdb) quit  
(base) kolin@mosaic:~/col7001/concurrency$ gcc -g -O6 sync.c  
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out  
Final counter = 2000000  
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out  
Final counter = 2000000  
(base) kolin@mosaic:~/col7001/concurrency$
```


Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <<http://gnu.org/licenses/gpl.html>>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<<https://www.gnu.org/software/gdb/bugs/>>.
Find the GDB manual and other documentation resources online at:
<<http://www.gnu.org/software/gdb/documentation/>>.

For help, type "help".

Type "apropos word" to search for commands related to "word"...

Reading symbols from ./a.out...

(gdb) list

```
1      #include <stdio.h>
2      #include <pthread.h>
3
4      int counter = 0; // shared variable
5
6      void* increment(void* arg) {
7          for (int i = 0; i < 1000000; i++) {
8              counter++; // data race: multiple threads update at the same time
9          }
10         return NULL;
11     }
```

(gdb) |



Q Search



09:13

23-09-2025

For bug reporting instructions, please see:
<<https://www.gnu.org/software/gdb/bugs/>>.
Find the GDB manual and other documentation resources online at:
<<http://www.gnu.org/software/gdb/documentation/>>.

For help, type "help".

Type "apropos word" to search for commands related to "word"...

Reading symbols from ./a.out...

(gdb) list

```
1      #include <stdio.h>
2      #include <pthread.h>
3
4      int counter = 0; // shared variable
5
6      void* increment(void* arg) {
7          for (int i = 0; i < 1000000; i++) {
8              counter++; // data race: multiple threads update at the same time
9          }
10         return NULL;

```

(gdb) disassemble increment

Dump of assembler code for function increment:

```
0x0000000000001250 <+0>:      endbr64
0x0000000000001254 <+4>:      addl    $0xf4240,0x2db6(%rip)      # 0x4014 <counter>
0x000000000000125e <+14>:     xor     %eax,%eax
0x0000000000001260 <+16>:     ret

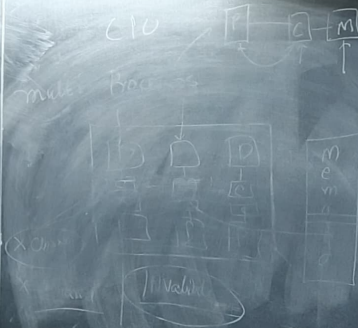
```

End of assembler dump.

(gdb)

Constant Folding
Dead Code Elim.

for $i = 1$ to N
 $Count = Count + 1$
 $Count = Count + N$



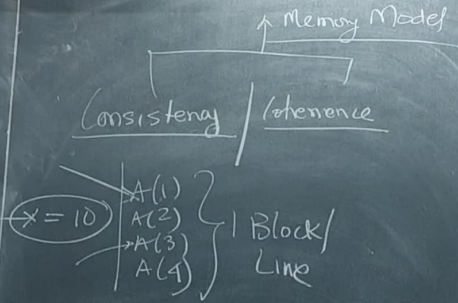
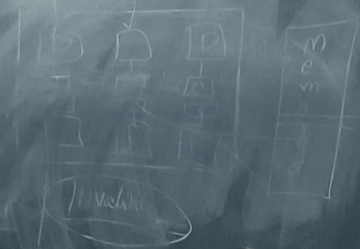
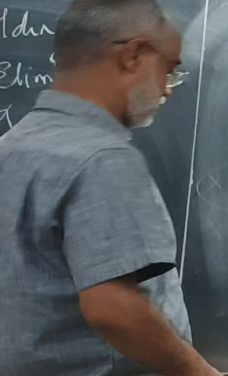
Consistency / Coherence

False Sharing

$x = 10$

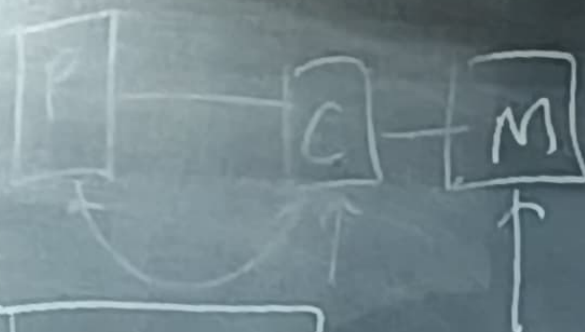
A(1)
A(2)
A(3)
A(4)

Block Line



False Sharing

CPU



Stored {

int a

Char b

{ xyz

xyz(0)

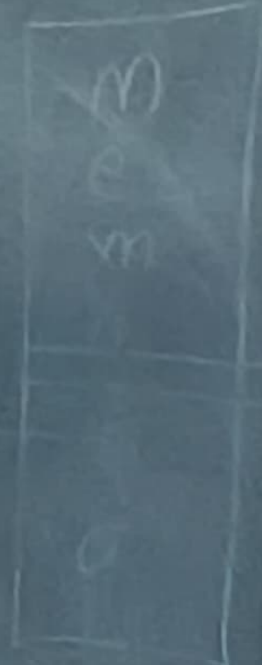
xyz(1)

xyz(2)

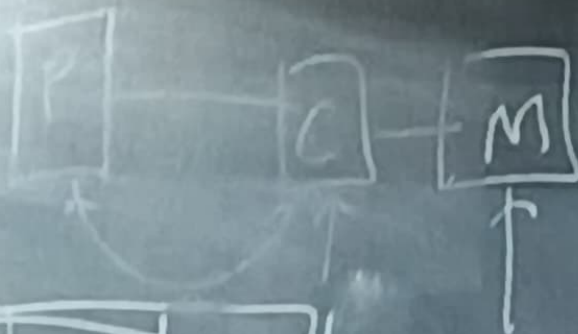
00	✓
01	✓
20	
21	
30	
31	

xyz

abc[10];



CPU



Stored {

int a

Char b

{ xyz

xyz

xyz(0)	00 ✓
	01 ✓
xyz(1)	20
xyz(2)	21 →
	30
	31

abc[10];

```
#include <stdio.h>
#include <pthread.h>

int x = 0, y = 0;
int r1, r2;

void* thread1(void* arg) {
    x = 1;           // write x
    r1 = y;          // read y
    return NULL;
}

void* thread2(void* arg) {
    y = 1;           // write y
    r2 = x;          // read x
    return NULL;
}

int main() {
    for (int i = 0; i < 1000000; i++) {
        x = y = 0; r1 = r2 = 0;
        pthread_t t1, t2;
        pthread_create(&t1, NULL, thread1, NULL);
        pthread_create(&t2, NULL, thread2, NULL);
        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
        printf("r1=%d\tr2=%d\n", r1, r2);
    }
}
```



```
#include <stdio.h>
#include <pthread.h>

#include <stdatomic.h>

atomic_int x = 0, y = 0;
int r1, r2;

/*
int x = 0, y = 0;
void* thread1(void* arg) {
    x = 1;           // write x
    r1 = y;          // read y
    return NULL;
}

void* thread2(void* arg) {
    y = 1;           // write y
    r2 = x;          // read x
    return NULL;
}
*/

void* thread1(void* arg) {
    atomic_store_explicit(&x, 1, memory_order_release);
    r1 = atomic_load_explicit(&y, memory_order_acquire);
    return NULL;
}
void* thread2(void* arg) {
    reorderF.c
```

*/

```
void* thread1(void* arg) {
    atomic_store_explicit(&x, 1, memory_order_release);
    r1 = atomic_load_explicit(&y, memory_order_acquire);
    return NULL;
}
```

```
void* thread2(void* arg) {
    atomic_store_explicit(&y, 1, memory_order_release);
    r2 = atomic_load_explicit(&x, memory_order_acquire);
    return NULL;
}
```

```
int main() {
    for (int i = 0; i < 1000000; i++) {
        x = y = 0; r1 = r2 = 0;
        pthread_t t1, t2;
        pthread_create(&t1, NULL, thread1, NULL);
        pthread_create(&t2, NULL, thread2, NULL);
        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
        if (r1 == 0 && r2 == 0) {
            printf("Reordering observed! in %dth Iteration\n", i);
            break;
        }
    }
}
```

(END)

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=0 r2=1

r1=1 r2=0

^C

(base) kolin@mosaic:~/col7001/concurrency\$ ls
a.out cacheSharing.c fs-v2.c reorderF.c reorderWithComm.c
cacheC.c fs.c reorder.c reorderSeqCons.c syncB.c
(base) kolin@mosaic:~/col7001/concurrency\$ less reorderF.c
(base) kolin@mosaic:~/col7001/concurrency\$ less reorderWithComm.c

```
#include <stdio.h>
#include <pthread.h>

int x = 0, y = 0;
int r1, r2;

void* thread1(void* arg) {
    x = 1;           // write x
    r1 = y;          // read y
    return NULL;
}

void* thread2(void* arg) {
    y = 1;           // write y
    r2 = x;          // read x
    return NULL;
}

int main() {
    for (int i = 0; i < 10000000; i++) {
        x = y = 0; r1 = r2 = 0;
        pthread_t t1, t2;
        pthread_create(&t1, NULL, thread1, NULL);
        pthread_create(&t2, NULL, thread2, NULL);
        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
        if (r1 == 0 && r2 == 0) {
            printf("Reordering observed!\n");
            break;
        }
    }
}
```

reorderWithComm.c

5


```
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=1    r2=0
```

^C

```
(base) kolin@mosaic:~/col7001/concurrency$ ls
a.out      cacheSharing.c  fs-v2.c  reorderF.c      reorderWithComm.c  sync.c  syncM.c
cacheC.c   fs.c            reorder.c  reorderSeqCons.c  syncB.c            syncL.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed!

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Constant Folding
Dead Code Elim.

$fs = 1$ — $\nearrow$
 $Count = Count + 1$
 $\rightarrow Count = Count + N$

```
r1=0 r2=1
r1=0 r2=1
r1=0 r2=1
r1=0 r2=1
r1=0 r2=1
r1=0 r2=1
r1=0 r2=1
r1=0 r2=1
r1=0 r2=1
r1=0 r2=1
r1=1 r2=0
^C
(base) kolin@mosaic:~/col7001/concurrency$ ls
a.out  cacheSharing.c  fs-v2.c  reorderF.c  reorderWithComm.c  sync.c  syncL.c
cacheC.c  fs.c  reorder.c  reorderSeqCons.c  syncB.c  syncL.c
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
(base) kolin@mosaic:~/col7001/concurrency$ less reorderWithComm.c
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderWithComm.c
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Reordering observed!
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderF.c
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Reordering observed! in 161107th Iteration
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Reordering observed! in 84865th Iteration
(base) kolin@mosaic:~/col7001/concurrency$
```

[illegible]

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderWithComm.c
(base) kolin@mosaic:~/col7001/concurrency$
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed!

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

```
Reordering observed! in 161107th Iteration
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

```
Reordering observed! in 84865th Iteration
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorder
```

```
(base) kotlinc@msaic:~/Code/2021/Concurrent/seq4$ ls -la
total 16
drwxr-xr-x 2 root root 4096 Nov 10 16:28
-rw-r--r-- 1 root root  117 Nov 10 16:28 reorder.c
-rw-r--r-- 1 root root  117 Nov 10 16:28 reorderF.c
-rw-r--r-- 1 root root  117 Nov 10 16:28 reorderSeqCons.c
-rw-r--r-- 1 root root  117 Nov 10 16:28 reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorder
```



```
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=1    r2=0
```

^C

```
(base) kolin@mosaic:~/col7001/concurrency$ ls
a.out      cacheSharing.c  fs-v2.c      reorderF.c      reorderWithComm.c  sync.c      syncM.c
cacheC.c   fs.c            reorder.c    reorderSeqCons.c  syncB.c            syncL.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed!

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed! in 161107th Iteration

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed! in 84865th Iteration

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorder
```

```
reorder.c      reorderF.c      reorderSeqCons.c  reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderSeqCons.c
```

```
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>

atomic_int x = 0, y = 0;
int r1, r2;

void* thread1(void* arg) {
    atomic_store_explicit(&x, 1, memory_order_seq_cst);
    r1 = atomic_load_explicit(&y, memory_order_seq_cst);
    return NULL;
}

void* thread2(void* arg) {
    atomic_store_explicit(&y, 1, memory_order_seq_cst);
    r2 = atomic_load_explicit(&x, memory_order_seq_cst);
    return NULL;
}

int main() {
    for (int i = 0; i < 1000000; i++) {
        atomic_store(&x, 0);
        atomic_store(&y, 0);
        r1 = r2 = 0;

        pthread_t t1, t2;
        reorderSeqCons.c
```

```
void* thread2(void* arg) {  
    atomic_store_explicit(&y, 1, memory_order_seq_cst);  
    r2 = atomic_load_explicit(&x, memory_order_seq_cst);  
    return NULL;  
}
```

```
int main() {  
    for (int i = 0; i < 1000000; i++) {  
        atomic_store(&x, 0);  
        atomic_store(&y, 0);  
        r1 = r2 = 0;  
  
        pthread_t t1, t2;  
        pthread_create(&t1, NULL, thread1, NULL);  
        pthread_create(&t2, NULL, thread2, NULL);  
        pthread_join(t1, NULL);  
        pthread_join(t2, NULL);  
  
        if (r1 == 0 && r2 == 0) {  
            printf("Reordering observed (i=%d)\n", i);  
            break;  
        }  
    }  
    return 0;  
}
```

(END)


```
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=1    r2=0
^C
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ls
```

```
a.out    cacheSharing.c  fs-v2.c    reorderF.c    reorderWithComm.c  sync.c    syncM.c
cacheC.c fs.c          reorder.c  reorderSeqCons.c  syncB.c          syncL.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed!

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed! in 161107th Iteration

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed! in 84865th Iteration

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorder
```

```
reorder.c    reorderF.c    reorderSeqCons.c  reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderSeqCons.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderSeqCons.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

```
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=0    r2=1
r1=1    r2=0
```

^C

(base) kolin@mosaic:~/col7001/concurrency\$ ls

```
a.out      cacheSharing.c  fs-v2.c  reorderF.c      reorderWithComm.c  sync.c  syncM.c
cacheC.c  fs.c          reorder.c  reorderSeqCons.c  syncB.c            syncL.c
```

(base) kolin@mosaic:~/col7001/concurrency\$ less reorderF.c

(base) kolin@mosaic:~/col7001/concurrency\$ less reorderWithComm.c

(base) kolin@mosaic:~/col7001/concurrency\$ gcc reorderWithComm.c

(base) kolin@mosaic:~/col7001/concurrency\$./a.out

Reordering observed!

(base) kolin@mosaic:~/col7001/concurrency\$ less reorderF.c

(base) kolin@mosaic:~/col7001/concurrency\$ gcc reorderF.c

(base) kolin@mosaic:~/col7001/concurrency\$./a.out

Reordering observed! in 161107th Iteration

(base) kolin@mosaic:~/col7001/concurrency\$./a.out

Reordering observed! in 84865th Iteration

(base) kolin@mosaic:~/col7001/concurrency\$ less reorder

```
reorder.c      reorderF.c      reorderSeqCons.c  reorderWithComm.c
```

(base) kolin@mosaic:~/col7001/concurrency\$ less reorderSeqCons.c

(base) kolin@mosaic:~/col7001/concurrency\$ gcc reorderSeqCons.c

(base) kolin@mosaic:~/col7001/concurrency\$./a.out

(base) kolin@mosaic:~/col7001/concurrency\$


```
#include <stdio.h>
#include <pthread.h>
#include <time.h>
```

```
#define N 1000000000
```

```
// --- Case 1: False Sharing ---
```

```
struct {
    int a;    // updated by thread 1
    int b;    // updated by thread 2
} shared;
```

```
// --- Case 2: Fixed with Padding ---
```

```
struct {
    int a;
    char pad1[64]; // avoid same cache line
    int b;
    char pad2[64];
} shared_padded;
```

```
void* t1(void* arg) {
    int mode = *(int*)arg;
    for (int j=0; j<N/10; j++){
        if (mode == 0) {
            for (int i = 0; i < N; i++) shared.a++;
        } else {
```

fs.c


```
#include <stdio.h>
#include <pthread.h>
#include <time.h>

#define N 1000000000

// --- Case 1: False Sharing ---
struct {
    int a;    // updated by thread 1
    int b;    // updated by thread 2
} shared;

// --- Case 2: Fixed with Padding ---
struct {
    int a;
    char pad1[64]; // avoid same cache line
    int b;
    char pad2[64];
} shared_padded;

void* t1(void* arg) {
    int mode = *(int*)arg;
    for (int j=0; j<N/10; j++){
        if (mode == 0) {
            for (int i = 0; i < N; i++) shared.a++;
        } else {
```

fs.c



Q Search



```

void* t1(void* arg) {
    int mode = *(int*)arg;
    for (int j=0; j<N/10; j++){
        if (mode == 0) {
            for (int i = 0; i < N; i++) shared.a++;
        } else {
            for (int i = 0; i < N; i++) shared_padded.a++;
        }
    }
    return NULL;
}

```

```

void* t2(void* arg) {
    int mode = *(int*)arg;
    if (mode == 0) {
        for (int i = 0; i < N; i++) shared.b++;
    } else {
        for (int i = 0; i < N; i++) shared_padded.b++;
    }
    return NULL;
}

```

```

double run_test(int mode) {
    struct timespec start, end;
    pthread_t x, y;
    :|

```



Search



```
}  
}  
return NULL;  
}  
  
void* t2(void* arg) {  
    int mode = *(int*)arg;  
    if (mode == 0) {  
        for (int i = 0; i < N; i++) shared.b++;  
    } else {  
        for (int i = 0; i < N; i++) shared_padded.b++;  
    }  
    return NULL;  
}
```

```
double run_test(int mode) {  
    struct timespec start, end;  
    pthread_t x, y;  
  
    clock_gettime(CLOCK_MONOTONIC, &start);  
  
    pthread_create(&x, NULL, t1, &mode);  
    pthread_create(&y, NULL, t2, &mode);  
    pthread_join(x, NULL);  
    pthread_join(y, NULL);  
}
```

∴



Search



09:50

23-09-2025

r1=1 r2=0

^C

```
(base) kolin@mosaic:~/col7001/concurrency$ ls
a.out      cacheSharing.c  fs-v2.c      reorderF.c      reorderWithComm.c  sync.c      syncM.c
cacheC.c   fs.c            reorder.c     reorderSeqCons.c  syncB.c            syncL.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed!

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderF.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed! in 161107th Iteration

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

Reordering observed! in 84865th Iteration

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorder
```

```
reorder.c      reorderF.c      reorderSeqCons.c  reorderWithComm.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less reorderSeqCons.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderSeqCons.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less fs
```

```
fs.c      fs-v2.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ less fs.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$
```



Search



09:50
23-09-2025

```
#define N 1000000000
```

```
// --- Case 1: False Sharing ---
```

```
struct {  
    int a;    // updated by thread 1  
    int b;    // updated by thread 2  
} shared;
```

```
// --- Case 2: Fixed with Padding ---
```

```
struct {  
    int a;  
    char pad1[64]; // avoid same cache line  
    int b;  
    char pad2[64];  
} shared_padded;
```

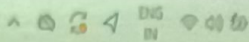
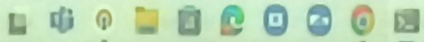
```
void* t1(void* arg) {  
    int mode = *(int*)arg;  
    for (int j=0; j<N/10; j++){  
        if (mode == 0) {  
            for (int i = 0; i < N; i++) shared.a++;  
        } else {  
            for (int i = 0; i < N; i++) shared_padded.a++;  
        }  
    }  
}
```

24,20

7%



Search



09:32
23-09-2025

```
// --- Case 1: False Sharing ---
```

```
struct {  
    int a;    // updated by thread 1  
    int b;    // updated by thread 2  
} shared;
```

```
// --- Case 2: Fixed with Padding ---
```

```
struct {  
    int a;  
    char pad1[64]; // avoid same cache line  
    int b;  
    char pad2[64];  
} shared_padded;
```

```
void* t1(void* arg) {  
    int mode = *(int*)arg;  
    for (int j=0; j<N/10; j++){  
        if (mode == 0) {  
            for (int i = 0; i < N; i++) shared.a++;  
        } else {  
            for (int i = 0; i < N; i++) shared_padded.a++;  
        }  
    }  
    return NULL;  
}
```

~@k

23,22

11%



Q Search




```
// --- Case 1: False Sharing ---
```

```
struct {  
    int a;    // updated by thread 1  
    int b;    // updated by thread 2  
} shared;
```

```
// --- Case 2: Fixed with Padding ---
```

```
struct {  
    int a;  
    char pad1[64]; // avoid same cache line  
    int b;  
    char pad2[64];  
} shared_padded;
```

```
void* t1(void* arg) {  
    int mode = *(int*)arg;  
    for (int j=0; j<N/1000; j++){  
        if (mode == 0) {  
            for (int i = 0; i < N; i++) shared.a++;  
        } else {  
            for (int i = 0; i < N; i++) shared_padded.a++;  
        }  
    }  
    return NULL;  
}
```

Z

23, 24

11%



Q Search



ENG

23-09-2025

09:52

```
a.out    cacheSharing.c  fs-v2.c    reorderF.c    reorderWithComm.c  sync.c    syncM.c
cacheC.c  fs.c              reorder.c  reorderSeqCons.c  syncB.c          syncL.c
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
(base) kolin@mosaic:~/col7001/concurrency$ less reorderWithComm.c
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderWithComm.c
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Reordering observed!
(base) kolin@mosaic:~/col7001/concurrency$ less reorderF.c
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderF.c
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Reordering observed! in 161107th Iteration
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
Reordering observed! in 84865th Iteration
(base) kolin@mosaic:~/col7001/concurrency$ less reorder
reorder.c    reorderF.c    reorderSeqCons.c  reorderWithComm.c
(base) kolin@mosaic:~/col7001/concurrency$ less reorderSeqCons.c
(base) kolin@mosaic:~/col7001/concurrency$ gcc reorderSeqCons.c
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
(base) kolin@mosaic:~/col7001/concurrency$ less fs
fs.c    fs-v2.c
(base) kolin@mosaic:~/col7001/concurrency$ less fs.c
(base) kolin@mosaic:~/col7001/concurrency$ gcc fs.c
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
^C
(base) kolin@mosaic:~/col7001/concurrency$ vi fs.c

(base) kolin@mosaic:~/col7001/concurrency$ gcc fs.c
```


fs.c:23:24: error: expected ';' before ')' token

```
23 |     for (int j=0;j<1j++){
```

```
(base) kolin@mosaic:~/col7001/concurrency$ vi fs.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ gcc fs.c
```

```
(base) kolin@mosaic:~/col7001/concurrency$ ./a.out
```

False sharing time: 3.665 sec

Fixed (padded) time: 1.867 sec

```
(base) kolin@mosaic:~/col7001/concurrency$
```



23-Sept-2025

Assembly version of last class code shown

Compiled it with no optimization

→ Now we will compile it with highest level of optimization
(without changing Semantics of code)

now same thread code give 2000 without lock
on -O6 level optimization (Highest)

→ loop completely gone [Removed by optimization]

→ by doing constant folding & Deadcode elimination

$\left\{ \begin{array}{l} \text{for } (i = 1 \dots n) \\ \quad \text{count}++ \end{array} \right\} \rightarrow \text{compile time constant}$

basically = $\text{count} = \text{count} + n$

~~So~~ So why should I do loop

this value is not gonna be used again

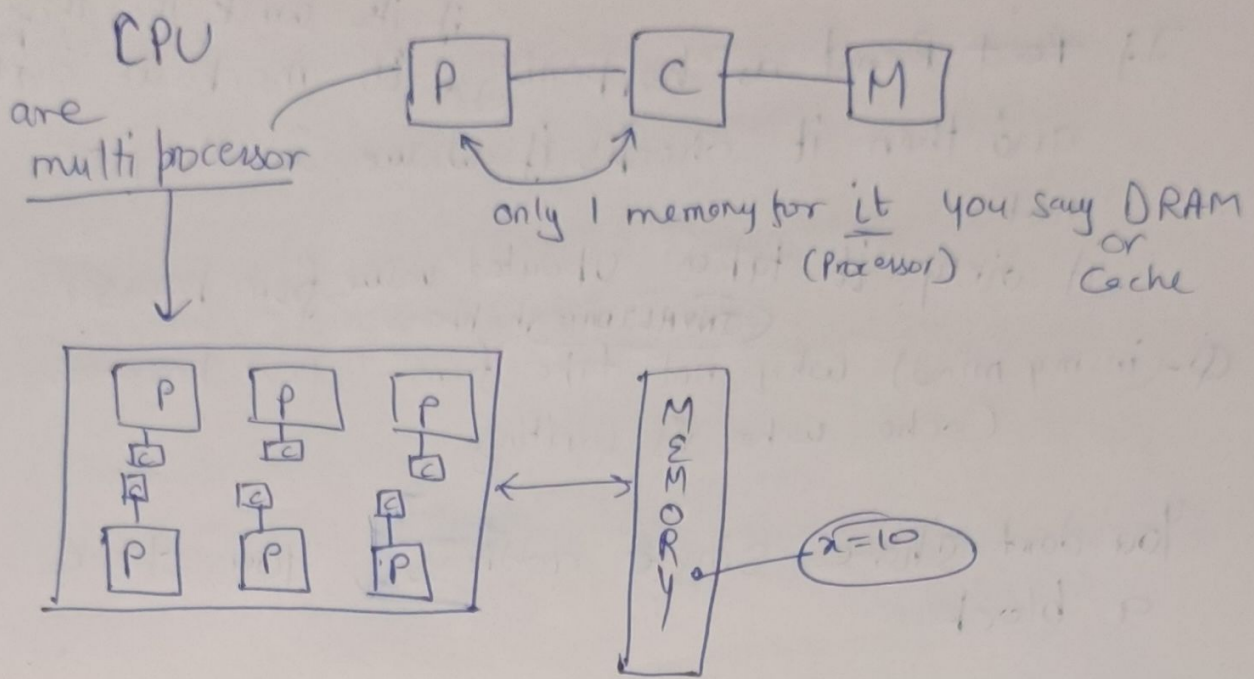
Removed loop, substituted n , no bug was just a side effect

[Some variant of optimization can be Question
in exam]

Volatile means compiler can not do optimization
but what optimization it can not do?

it can do const folding

→ function isn't multithreaded main is multithreaded

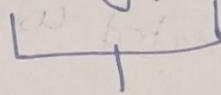


Now there is this problem of Consistency/Coherence

Key idea every processor should see same value

Coherence - Update must propagate to all processor in finite amount of time

Consistency/coherence



Memory Model

~~when you have concurrent processor~~

How it works?

If we have a memory location $x=10$ Processor need that line, it will pick up that specific block and send it to cache,

$\times$ Cache becomes shared if some other also brings in x -shared

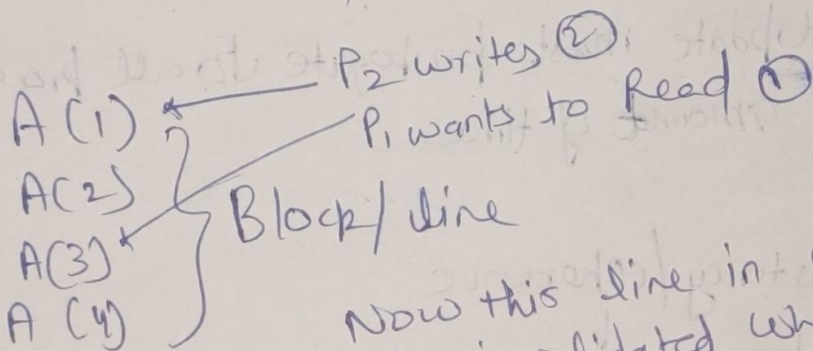
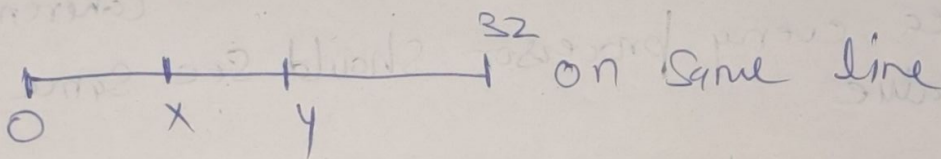
If ~~Read~~ Read is happening ^{if its write then there is problem} it marks it dirty and then it checks if value is dirty or not

if dirty it takes updated value from processor

Q-(in my mind) why not take from follow processors
Cache who is writing
INVALIDATE happens

You dont share single address, you share a block

P₁ — Accessing X
P₂ — Accessing Y



Now this line in cache of P₁ gets invalidated when P₂ reads

So we have alot of invalidate signal going around

Prog is not doing anything shared
Hardware performing shared operations,
programmer doesn't know
therefore we get lots of Invalidate signal
making prog ~~is~~ slow (False sharing)

How we do it?

- use multiple cache lines

Struct {

5
or
8 { int a
~~int b~~
char b

xyz

xyz

(tight code)

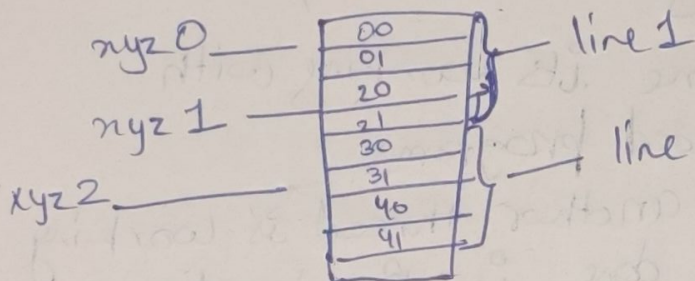
Non padded - 50 bytes

padded - 80 bytes

(Assuming word is 4 bytes)

abc[10];

tight code saves you 30 bytes

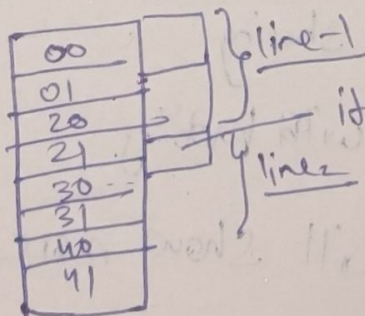


Cache is able to take 4 words

So 00 01 20 21 on line 1

30 31 40 41 on line 2

but if it was 5 bytes



write
if this has ~~invalidate~~ it will cause invalidate
20 cache line also

If you have shared variable, you just ensure rights are atomic

using `atomic_int x, y;`

`atomic_store_explicit(&x, 1, memory_order)`

an Alternative of locks

but even atomic does not fix it in
iteration 161107 we observed 0,0
(Check code from PPT)

A very hard bug to catch
Nothing wrong with ~~at~~ code,

C compiler assume its working with
Single threaded programs

it does not know another thread is working
so what it has done is - Reordered your
instructions

- Sequential Consistency model

→ till now we worked on Relaxed memory
(check ppt for code) model

(Code check this) { Case 1 - false sharing
Case 2 - fixed with padding

one with padded will show more time

build a library of your own for CP